BAHIR DAR INSTITUTE OD TECHNOLOGY

FACULTY OF COMPUTING

DEPARTEMENT OF SOFTWARE ENGINEERING

COURSE: COMPILER DESIGN

INDIVIDUAL ASSIGNMENT

NAME..... DANIEL ANDUALEM

ID 1505662

SECTION.....B

SUBMITTED TO : WENDMU

Table of Contents

Assignment 02

Left-Factoring and Its Relation to Lexing in Compiler Design

1. Introduction

Compiler design is a fundamental area of computer science that focuses on translating high-level programming languages into machine-executable code. One of the most critical phases of a compiler is syntax analysis (parsing), where the grammatical structure of a program is checked against a formal grammar. For efficient and correct parsing, the grammar used by the compiler must be free from ambiguity and suitable for deterministic parsing techniques. In this context, left-factoring plays an essential role in transforming context-free grammars into forms that can be easily processed by predictive parsers.

Left-factoring is a grammar transformation technique primarily used to eliminate common prefixes in production rules. These common prefixes can cause confusion for top-down parsers, particularly LL(1) parsers, which rely on a single token of lookahead to make parsing decisions. Without left-factoring, a parser may require backtracking or additional lookahead, leading to inefficiency and increased complexity. By restructuring grammar rules to factor out shared prefixes, left-factoring ensures that the parser can make deterministic choices, improving both performance and reliability.

In addition to its importance in parsing, left-factoring also has a strong relationship with lexical analysis (lexing). Lexers convert streams of characters into tokens, often using deterministic finite automata (DFA). Ambiguous or overlapping token patterns can lead to conflicts during token recognition. Applying left-factoring principles in lexical design helps resolve such ambiguities and ensures correct token classification. This assignment explores the concept of left-factoring, its process, importance in parsing, and its close relationship with lexing in compiler design.

In compiler design, left-factoring is a grammar transformation technique used to eliminate ambiguity in context-free grammars (CFGs) and make them suitable for predictive parsing.

Predictive parsing reads input from left to right and produces a leftmost derivation of the input string. Ambiguity or common prefixes in production rules can confuse the parser, as it may not know which production to apply without lookahead. Left-factoring ensures that the parser can make a deterministic choice.

Key Points of Left-Factoring

- Helps top-down parsers, especially LL(1) parsers**.**
- Eliminates the need for backtracking.
- Simplifies parser implementation by making grammar deterministic.

Left-factoring is the process of restructuring a grammar to factor out common prefixes among the productions of a non-terminal.

If a non-terminal `A` has productions:

A→αβ1|αβ2A

where α is a common prefix, left-factoring rewrites the grammar as

A→αA’A , A'A→αA’ A’→β1|β2A' , 2A’→β1|β2

Explanation

- The parser first matches α.
- After matching α, it chooses between `β1` or `β2` using the next token.
- This ensures that single-token lookahead is sufficient.

2. Example of Left-Factoring

Consider the grammar

S → if E then S else S | if E then S

Observation:

- Both productions start with `if E then S`.
- This common prefix makes predictive parsing ambiguous, as the parser cannot decide which production to use after reading `if E then S`.

Left-Factored Grammar

After left-factoring

S → if E then S S’

S′ → else S | ε

Explanation:

- `if E then S` is factored out as the common prefix.
- A new non-terminal `S'` handles the remaining alternatives either `else S` or nothing (ε).
- The parser can now predictively choose the correct production after matching the prefix.

Benefits of This Example

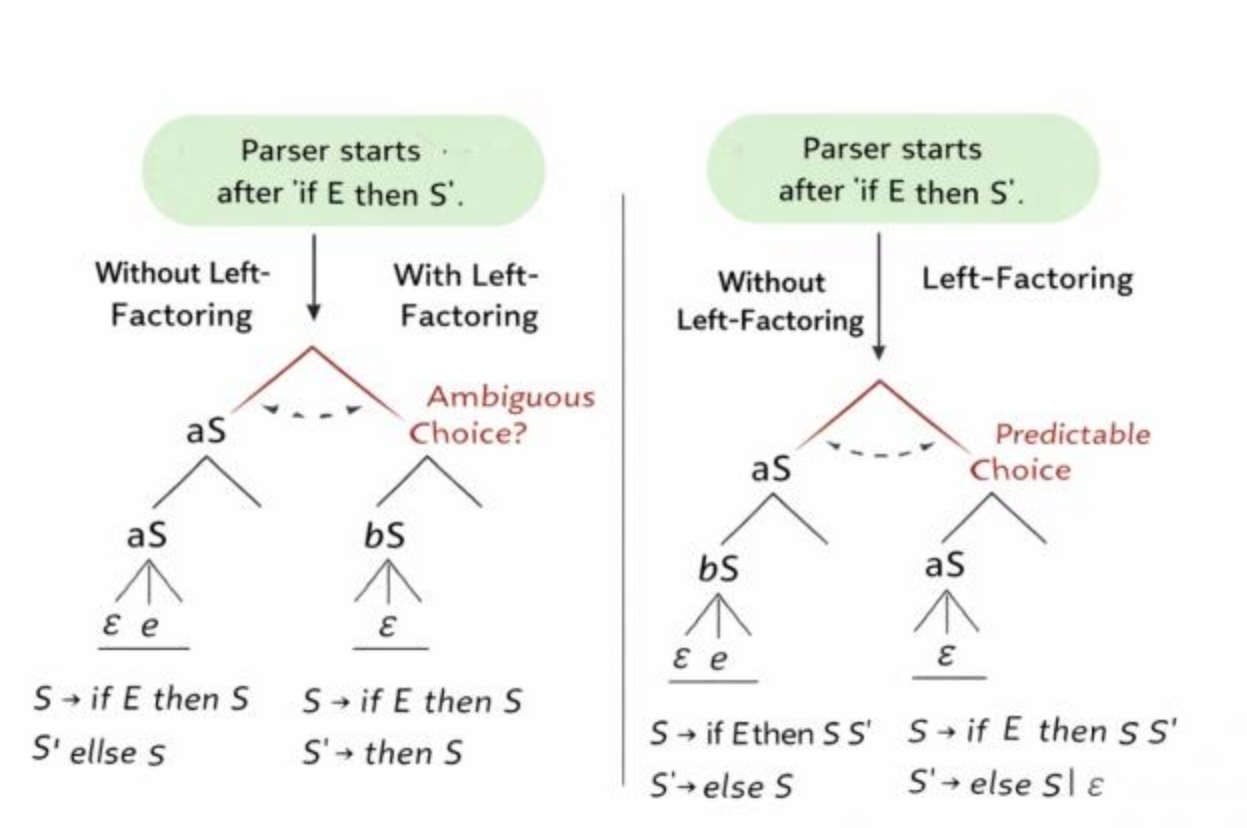
- **Ambiguity removed** The parser knows exactly what to expect after `if E then S`.
- **LL(1) compatible:** Only one token of lookahead is needed.
- **Simpler implementation:** Avoids backtracking in top-down parsers.

Observations

1. Left-factoring is especially important when common prefixes occur in productions.
2. This technique improves grammar readability and ensures deterministic parsing.
3. It is widely used in both parser generation and lexer design, as shared prefixes in tokens can also cause ambiguity.

3. Importance in Parsing

1. Predictive Parsing Compatibility
    - Predictive parsers rely on single-token look ahead.
    - Ambiguous choices at the start of productions prevent the parser from deciding which rule to apply.
    - Left-factoring resolves this by factoring out common prefixes.
2. Simplifies Parser Implementation
    - Avoids complex backtracking logic.
    - Improves parser performance and maintainability.



4. Left-Factoring Process

Steps of Left-Factoring

Given a grammar with rules:

```
A → αβ1 | αβ2
```

Step 1 Identify the common prefix (α).
Step 2 Factor out the prefix:

A → αA'

A' → β1 | β2

Example

Original Rule

stmt → if E then S else S | if E then S

Left-Factored

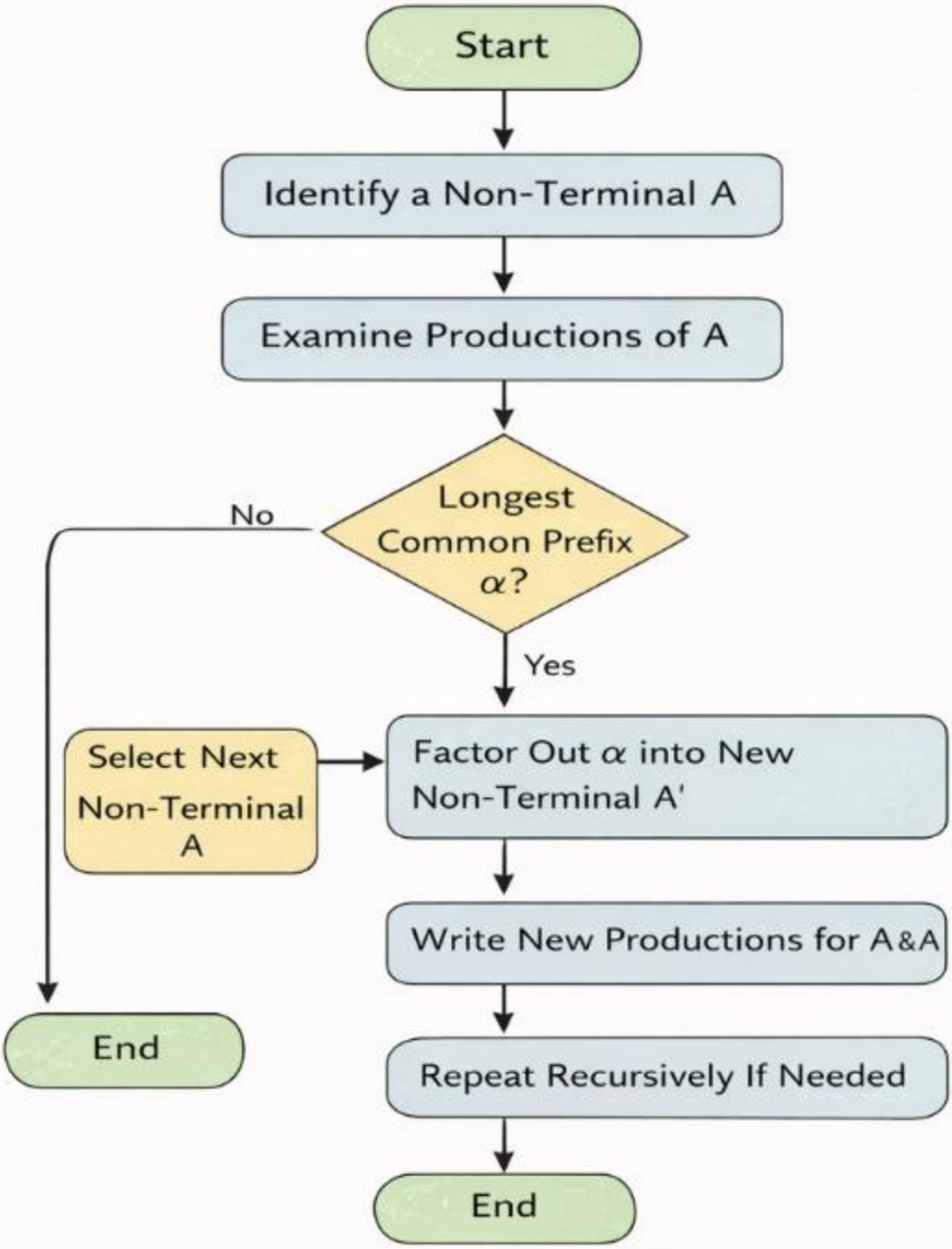
```
stmt → if E then S stmt'

stmt' → else S | ε
```

Algorithm for Left-Factoring

Step-by-step Algorithm

1. For each non-terminal `A`, examine all its productions.
2. Find the longest common prefix among alternatives.
3. Factor out the common prefix as a new non-terminal `A'`.
4. Replace original productions with the factored form.
5. Repeat recursively if needed.

Left-Factoring Algorithm



5. Relation to Lexing

Lexing Overview

- Lexical analysis converts a stream of characters into tokens.
- DFA (Deterministic Finite Automaton) is commonly used for lexing.

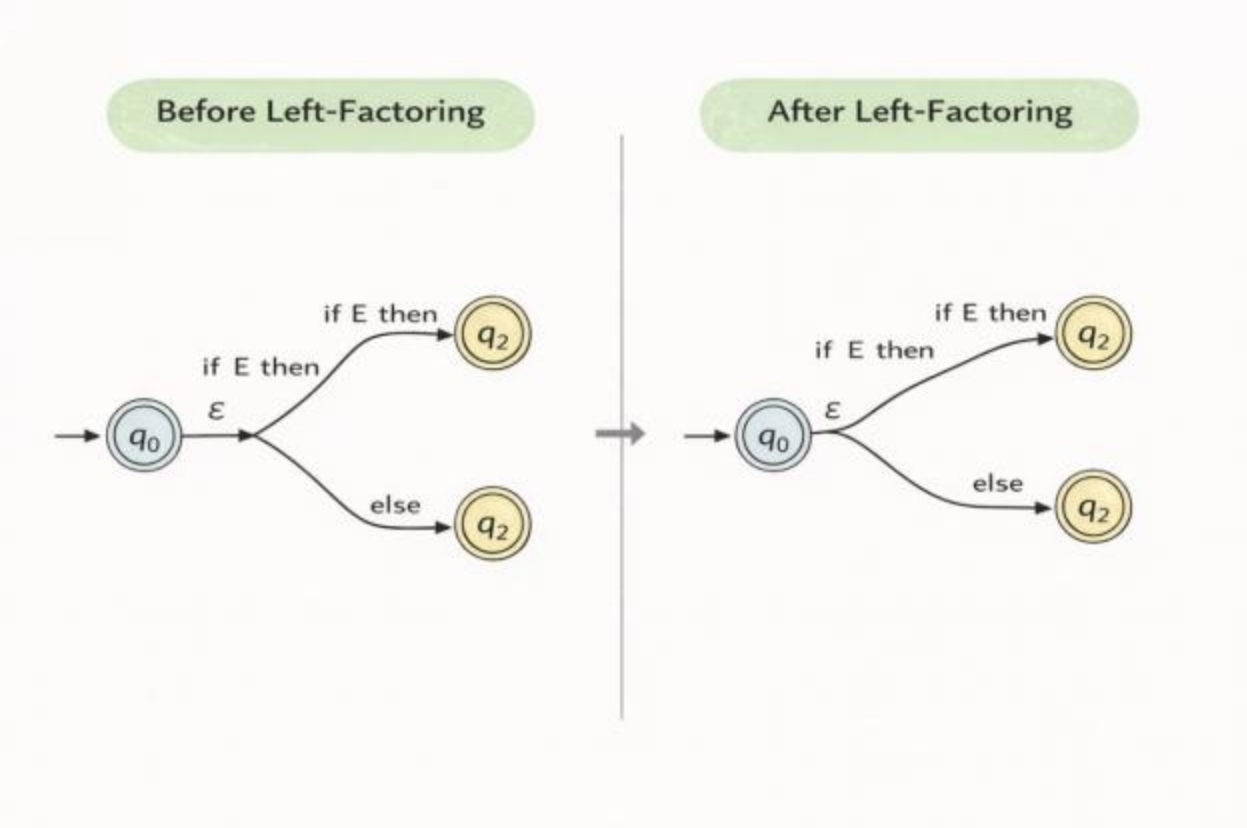
Left-Factoring in Lexical Context

- Ambiguous token patterns can cause lexer conflicts.
- Left-factoring ensures that the DFA can decide the correct token using the first character(s).

Example

```
Keywords: if, int

Identifier: i*
```

Without left-factoring, the lexer might fail to distinguish between `if` and an identifier starting with `i`.



6. Benefits of Left-Factoring

1. Ensures LL(1) compatibility for predictive parsers.
2. Reduces backtracking and parsing errors.
3. Simplifies lexer design for overlapping patterns.
4. Makes grammars cleaner and easier to maintain.
5. Preserves the language generated by the grammar.

7. Examples in Lexing

Keyword vs Identifier Example

Grammar before left-factoring:

```
token → if | identifier
```

After left-factoring

```
token → i token'

token' → f | [a-z0-9]*
```

Numeric Literals Example

Grammar before left-factoring:

```
num → 0 | 0[0-7]* | [1-9][0-9]*
```

Left-factored DFA ensures **correct token selection**.

8. Practical Considerations

- Ensure all common prefixes are identified.
- Recursive left-factoring may be required for nested common prefixes.
- Over-factoring can increase grammar complexity, so factor judiciously.
- Test grammars with edge cases to ensure correct parsing.

Common Pitfalls

1. Forgetting ε in the factored non-terminal.
2. Overlooking indirect common prefixes in multi-level productions.
3. Not updating parser tables after grammar modification.

9. Conclusion

Left-factoring is a vital technique in compiler design that significantly improves the effectiveness and determinism of both parsing and lexical analysis. By eliminating common prefixes in grammar productions, left-factoring enables predictive parsers—especially LL(1) parsers—to operate efficiently using only single-token lookahead. This transformation removes ambiguity, reduces the need for backtracking, and simplifies the overall parser implementation, making grammars easier to understand and maintain.

Throughout this assignment, the concept of left-factoring has been examined in detail, including its definition, process, and practical application through examples such as conditional statements and token recognition scenarios. The discussion highlights how left-factoring preserves the language generated by the original grammar while restructuring it into a form suitable for deterministic parsing. Furthermore, the relationship between left-factoring and lexing demonstrates that similar principles apply at both the syntactic and lexical levels of a compiler